

Dispensador inteligente de comida para gatos

Ing. Matías Nahuel Durante

Carrera de Especialización en Sistemas Embebidos

Director: Esp. Ing. Santiago Bualó (FIUBA)

Jurados:

Esp. Ing. Martin Duarte (FIUBA)
Esp. Ing. Santiago Escribá (FIUBA)
Esp. Ing. Lucas Monzón Languasco (FIUBA)

Ciudad de Córdoba, agosto de 2026

Resumen

La presente memoria describe el desarrollo de un sistema inteligente de dispensado de comida para gatos. La finalidad del trabajo es mejorar el bienestar animal mediante la automatización del suministro de alimento y el monitoreo del consumo a través de un software. Para ello se emplea un microcontrolador de alto desempeño, sensores de pesaje y módulos de comunicación inalámbrica, lo que permitió obtener una solución que combina el cuidado de mascotas con los avances de los sistemas embebidos e Internet de las Cosas (IoT).

El desarrollo incluye el diseño del hardware, la implementación del software de control y la validación del sistema mediante pruebas funcionales. Se aplicaron conocimientos en electrónica, programación de microcontroladores y protocolos de comunicación, con el fin de obtener un producto confiable, autónomo y con potencial de aplicación comercial.

Agradecimientos

A mi novia, mi familia, amigos y compañeros.

A mi director de trabajo, Esp. Ing. Santiago Bualó, por compartir sus conocimientos y orientación a lo largo de la especialización.

A mis colegas ingenieros, por el apoyo constante, los valiosos consejos y las sugerencias técnicas brindadas durante cada etapa del desarrollo de este trabajo.

Índice general

Resumen	I
1. Introducción general	1
1.1. Introducción a los sistemas de alimentación automatizada	1
1.1.1. Alimentación y bienestar animal	1
1.1.2. Domótica aplicada al cuidado de mascotas	1
1.2. Contexto y motivación	2
1.3. Estado del arte	2
1.3.1. PetSafe Smart Feed	2
1.3.2. PETLIBRO Granary Wi-Fi	3
1.3.3. Análisis del estado del arte y aportes del desarrollo	4
1.4. Objetivos del trabajo	5
2. Introducción específica	7
2.1. Componentes de hardware del sistema	7
2.1.1. Microcontrolador STM32F446RE	7
2.1.2. Módulo ESP32	8
2.1.3. Celda de carga y módulo HX711	8
2.1.4. Motor paso a paso y driver A4988	9
2.1.5. Módulo RTC DS3231	9
2.2. Software y entorno de desarrollo	10
2.2.1. STM32CubeIDE	10
2.2.2. FreeRTOS	10
2.2.3. Doxygen	10
2.2.4. PlatformIO	10
2.2.5. Flutter	11
2.2.6. Lenguaje de programación C	11
2.3. Protocolos de comunicación	11
2.3.1. Protocolo I2C	11
2.3.2. Protocolo UART	11
2.3.3. Bluetooth Classic	11
3. Diseño e implementación	13
3.1. Arquitectura general del sistema	13
3.1.1. Bloque de sensado	14
3.1.2. Bloque de control	14
3.1.3. Bloque de actuación	14
3.1.4. Bloque de comunicación con el usuario	14
3.1.5. Interacción entre hardware y software	15
3.2. Arquitectura de hardware	15
3.2.1. Gestión de alimentación	15
3.2.2. Interconexión de componentes	16

3.2.3.	Análisis de potencia	18
3.3.	Diseño del firmware	19
3.3.1.	Organización del firmware	19
3.3.2.	Módulo orquestador	20
3.3.3.	Módulo de periféricos	22
3.3.4.	Módulo de pesaje	22
3.3.5.	Módulo de gestión de horarios	22
3.3.6.	Módulo de gestión de almacenamiento	23
3.4.	Comunicaciones e interfaces	23
3.4.1.	Módulo de comunicación	23
3.4.2.	Interfaz de usuario	25
3.5.	Integración hardware-software	25
4.	Ensayos y resultados	29
4.1.	Plan de ensayos	29
4.1.1.	Metodología	29
4.1.2.	Instrumentos utilizados	29
4.1.3.	Criterios de aceptación	30
4.2.	Ensayos de hardware	30
4.2.1.	Celda de carga y módulo HX711	30
4.2.2.	Motor paso a paso NEMA 17 y driver A4988	31
4.2.3.	Módulo RTC DS3231	31
4.2.4.	Pulsador de dispensado manual	32
4.2.5.	LED de estado del sistema	32
4.3.	Ensayos de firmware	32
4.3.1.	Máquina de estados y módulo orquestador	32
4.3.2.	Gestión de tareas concurrentes en FreeRTOS	33
4.3.3.	Protocolo de comunicación y timing de telemetría	33
4.4.	Ensayos de integración	34
4.5.	Análisis de resultados	35
4.5.1.	Cumplimiento de criterios de aceptación	35
4.5.2.	Limitaciones identificadas	36
4.5.3.	Desvíos respecto de lo planificado	36
5.	Conclusiones	37
5.1.	Conclusiones generales	37
5.2.	Próximos pasos	37
	Bibliografía	39

Índice de figuras

1.1. Dispensador automático PetSafe <i>Smart Feed</i>	3
1.2. Dispensador automático PETLIBRO <i>Granary Smart Feeder</i>	4
2.1. Placa de evaluación Nucleo de STM	7
2.2. Módulo ESP32	8
2.3. Celda de carga y módulo HX711	8
2.4. Motor paso a paso NEMA 17 y driver A4988	9
2.5. Módulo RTC DS3231	10
3.1. Diagrama en bloques del sistema.	13
3.2. Diagrama de capas hardware-software del sistema.	15
3.3. Interconexión entre la fuente de alimentación y el microcontrolador.	16
3.4. Esquema de conexiones del microcontrolador.	17
3.5. Interconexión entre el microcontrolador, el driver A4988 y el motor NEMA 17.	17
3.6. Diagrama en bloques de los subsistemas de software.	19
3.7. Diagrama de estados del dispensador.	20
3.8. Diagrama de secuencia del ciclo de dispensado.	21
3.9. Arquitectura de comunicación del sistema.	24
3.10. Diagrama de navegación de la aplicación.	25
3.11. Diagrama de integración hardware-software.	26
3.12. Diagrama de secuencia de la operación de tara remota.	27
4.1. Lecturas sucesivas del sistema de pesaje registradas en la terminal serie.	31
4.2. Captura del bus I ² C entre el STM32 y el DS3231 obtenida con el analizador lógico.	32
4.3. Registro de la comunicación JSON entre la aplicación y el microcontrolador.	34
4.4. Interfaz de la aplicación durante los ensayos de integración.	35

Índice de tablas

1.1. Comparación de características entre productos del estado del arte y el sistema desarrollado	5
3.1. Asignación de pines del microcontrolador STM32F446RE.	18
3.2. Consumo máximo de cada componente del sistema.	18
3.3. Estructura de los mensajes del protocolo de comunicación.	24
4.1. Criterios de aceptación por módulo.	30
4.2. Resultados de las pruebas unitarias de la máquina de estados. . . .	33
4.3. Resultados esperados vs. obtenidos en los ensayos de integración. .	34

A mi novia Sofía, por su compañía durante la cursada de la especialización y su apoyo constante durante el desarrollo del trabajo.

A mi familia, por el amor y el orgullo que sienten en cada pequeño paso que doy en dirección de lo que elegí para mi vida: la electrónica.

A todos ellos, ¡muchas gracias!

Capítulo 1

Introducción general

En este capítulo se presentan los conceptos fundamentales relacionados con los sistemas de alimentación automatizada para mascotas, el análisis del estado del arte, la motivación técnica y los objetivos que rigen el presente trabajo.

1.1. Introducción a los sistemas de alimentación automatizada

El cuidado de mascotas en el hogar ha evolucionado significativamente gracias al avance de los sistemas embebidos y la electrónica de bajo costo. En particular, la alimentación de animales domésticos representa una de las principales preocupaciones de los propietarios, especialmente ante jornadas laborales extensas o ausencias prolongadas del hogar que dificultan el cumplimiento de rutinas nutricionales estrictas. La automatización de este proceso permite asegurar que los animales reciban la cantidad de alimento adecuada en los horarios correctos, independientemente de la presencia de sus dueños.

1.1.1. Alimentación y bienestar animal

Uno de los principales desafíos en el cuidado de gatos domésticos es garantizar una alimentación regular y controlada. Una rutina alimenticia inadecuada puede derivar en patologías frecuentes como la obesidad felina, la diabetes y los trastornos digestivos [1], todas ellas asociadas a variaciones en la cantidad o los horarios de ingesta.

La medicina veterinaria establece que la ración diaria debe determinarse en función del peso, la edad y el estado de salud del animal, y distribuirse en momentos fijos del día para evitar ansiedad y alteraciones del comportamiento [2]. Conocer si el animal efectivamente consumió el alimento dispensado agrega una dimensión diagnóstica que los sistemas de alimentación tradicionales no contemplan.

1.1.2. Domótica aplicada al cuidado de mascotas

La domótica, entendida como la aplicación de tecnologías de automatización y control al entorno doméstico, ha experimentado un crecimiento sostenido en los últimos años [3]. La disponibilidad de microcontroladores de bajo costo, módulos de comunicación inalámbrica y sensores de alta precisión ha permitido extender estas soluciones a ámbitos que antes resultaban inaccesibles para el usuario común.

En el área específica del cuidado de mascotas, esta tendencia se traduce en dispositivos capaces de automatizar tareas rutinarias como la alimentación, la hidratación y el monitoreo de actividad, integrándose en muchos casos a plataformas de Internet de las Cosas (IoT) que permiten el acceso remoto desde dispositivos móviles.

1.2. Contexto y motivación

La motivación principal de este trabajo surge de la necesidad de contar con un sistema de alimentación para gatos que no solo automatice la entrega de comida, sino que también brinde información confiable sobre el comportamiento alimenticio de la mascota. Esta información resulta valiosa tanto para el propietario como para el veterinario, ya que permite detectar anomalías en el consumo que podrían indicar alteraciones en el estado de salud del animal.

Por otra parte, la disponibilidad de microcontroladores de alta capacidad, módulos de comunicación inalámbrica y sensores de precisión a precios accesibles representa una oportunidad para desarrollar soluciones tecnológicas que superen las limitaciones de los productos comerciales existentes, particularmente en lo que respecta a la incorporación de mecanismos de monitoreo del consumo real del alimento. Este aspecto resulta fundamental para satisfacer la necesidad planteada de obtener información confiable sobre los hábitos alimenticios de la mascota.

1.3. Estado del arte

En esta sección se presentan algunos productos disponibles en el mercado que abordan problemáticas similares a la planteada en este trabajo. Se realiza un análisis de sus características técnicas, limitaciones y diferencias respecto de la solución desarrollada.

1.3.1. PetSafe Smart Feed

PetSafe Smart Feed [4] es uno de los dispensadores automáticos para mascotas más difundidos en el mercado internacional. El dispositivo cuenta con conectividad Wi-Fi de 2,4 GHz y se gestiona mediante la aplicación móvil My PetSafe [5], compatible con iOS y Android. Permite programar hasta doce raciones diarias con tamaños que van desde 1/8 de taza (29 ml) hasta 4 tazas (946 ml) por comida, y dispone de una capacidad de almacenamiento de 24 tazas (5 678 ml) de alimento seco o semihúmedo. La aplicación envía notificaciones ante nivel bajo de alimento, depósito vacío o pérdida de conectividad. La figura 1.1 muestra el dispositivo, que se comercializa a un precio de USD 120,69.

Sin embargo, el sistema no cuenta con ningún mecanismo de pesaje del plato, por lo que no es posible determinar si el animal consumió efectivamente el alimento entregado. El historial disponible en la aplicación refleja únicamente los eventos de dispensado, y el funcionamiento depende de manera permanente de una conexión a Internet.



FIGURA 1.1. Dispensador automático PetSafe *Smart Feed*¹.

1.3.2. PETLIBRO Granary Wi-Fi

El modelo PETLIBRO Granary [6] con conectividad Wi-Fi es un dispensador automático de alimento seco para gatos y mascotas de tamaño pequeño o mediano. Soporta redes de 2,4 GHz y 5 GHz, y permite configurar hasta diez comidas diarias con porciones ajustables de 20 ml, sobre una capacidad total de 5 litros. Su mecanismo de dispensado por paletas de distribución uniforme reduce la ocurrencia de atascos, y el sistema incorpora sellado cuádruple con bolsa desecante para preservar la frescura del alimento. Cuenta con batería de respaldo ante cortes de energía. La figura 1.2 muestra el dispositivo, que se comercializa a un precio de USD 89,99 en el mercado estadounidense.

Al igual que el producto anterior, no incorpora un sistema de monitoreo del consumo real. El historial disponible en la aplicación registra únicamente los eventos de dispensado, sin ningún sensor que permita verificar si el animal ingirió efectivamente la ración entregada.

¹<https://www.petsafe.com/es/p/alimentador-de-mascotas-automatico-e-inteligente-smart-feed/PFD19-16861/>.



FIGURA 1.2. Dispensador automático PETLIBRO *Granary Smart Feeder*².

1.3.3. Análisis del estado del arte y aportes del desarrollo

Del análisis de los productos presentados se concluye que, si bien existen soluciones que automatizan la entrega de alimento con conectividad inalámbrica y control remoto, ninguna contempla la medición directa del consumo por parte del animal. Esta limitación impide obtener información confiable sobre el comportamiento alimenticio de la mascota, que es el dato de mayor valor para la detección temprana de alteraciones en su estado de salud. Adicionalmente, ambos productos analizados dependen de manera permanente de una conexión a Internet para operar, lo que los hace vulnerables ante cortes de conectividad o energía.

El sistema desarrollado en este trabajo incorpora como principal diferencial un módulo de pesaje del plato basado en una celda de carga HX711 [7], que permite determinar con precisión si el alimento dispensado fue consumido. La gestión autónoma de horarios se logra mediante un reloj de tiempo real (RTC), lo que elimina la dependencia de Internet. La comunicación con el usuario se realiza a través de Bluetooth con una aplicación en PC, y la gestión eficiente de tareas concurrentes se implementa sobre FreeRTOS [8]. La tabla 1.1 presenta una comparación de las características principales entre los productos analizados y el sistema desarrollado.

²<https://petlibro.com/products/petlibro-5g-wifi-automatic-pet-feeder>.

TABLA 1.1. Comparación de características entre productos del estado del arte y el sistema desarrollado.

Característica	PetSafe Smart Feed	PETLIBRO Granary	Sistema desarrollado
Conectividad	Wi-Fi 2,4 GHz	Wi-Fi 2,4/5 GHz	Bluetooth
Raciones programables	Hasta 12/día	Hasta 10/día	Por horario (RTC)
Pesaje del plato	No	No	Sí
Dependencia de Internet	Permanente	Permanente	No requerida
Batería de respaldo	No	Sí	Sí
Dispensado manual	No informado	No informado	Sí (pulsador físico)
Precio aproximado	USD 120,69	USD 89,99	USD 65,00

Entre los aportes descritos, el módulo de pesaje del plato representa el diferencial más significativo, ya que ninguno de los productos comerciales analizados es capaz de determinar si el animal consumió efectivamente el alimento entregado. La operación autónoma ante cortes de conectividad y el bajo costo de implementación con componentes disponibles en el mercado local complementan esta ventaja y conforman una solución con potencial de aplicación tanto doméstica como clínica.

1.4. Objetivos del trabajo

El objetivo del trabajo es desarrollar un prototipo funcional de un dispensador inteligente de alimento seco para gatos, capaz de automatizar la entrega de alimento en horarios programados, registrar el consumo real del animal mediante un sistema de pesaje y operar de forma autónoma ante cortes de conectividad o energía eléctrica. Es un dispositivo orientado a propietarios de gatos que deseen automatizar la alimentación de su mascota y obtener información confiable sobre sus hábitos de consumo. Para ello, el sistema debe satisfacer las siguientes necesidades:

- Automatizar la entrega de alimento en horarios programados.
- Registrar el consumo real del animal mediante el pesaje del plato.
- Operar de forma autónoma ante cortes de conectividad o energía eléctrica.
- Permitir la configuración de horarios y la consulta del historial de eventos por parte del usuario.
- Permitir un dispensado manual mediante acción del usuario.

Capítulo 2

Introducción específica

En este capítulo se describen los componentes de hardware, el software y los protocolos de comunicación utilizados para el desarrollo del sistema.

2.1. Componentes de hardware del sistema

En esta sección se describen los componentes electrónicos y módulos que conforman el hardware del sistema, entre ellos el microcontrolador, el motor de dispensado, la celda de carga y los módulos de comunicación y temporización.

2.1.1. Microcontrolador STM32F446RE

El microcontrolador empleado fue el modelo STM32F446RE de STMicroelectronics, que gestionó el dispensado de alimento, la lectura de sensores y la comunicación con los módulos periféricos.

Se seleccionó el STM32F446RE por su capacidad de procesamiento, su frecuencia de trabajo de hasta 180 MHz y su variedad de interfaces de comunicación, entre ellas UART [9] (*Universal Asynchronous Receiver-Transmitter*) e I²C [10] (*Inter-Integrated Circuit*). Estas interfaces permitieron consolidar en un solo dispositivo las funciones de control del dispensado, lectura de sensores y comunicación con los módulos periféricos. Su compatibilidad con FreeRTOS facilitó además la organización del firmware en tareas concurrentes.

Para el desarrollo de este trabajo se utilizó la placa de evaluación Nucleo-F446RE [11], que puede observarse en la figura 2.1 y que integra el módulo ST-LINK necesario para la programación y depuración del microcontrolador.

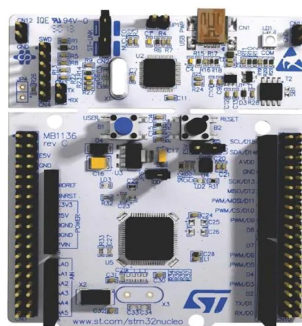


FIGURA 2.1. Placa de evaluación Nucleo de STM¹.

¹<https://www.st.com/en/evaluation-tools/nucleo-f446re.html>.

2.1.2. Módulo ESP32

Para la comunicación inalámbrica con el usuario se utilizó un módulo ESP32, que actuó como puente (*bridge*) entre el microcontrolador STM32 y la aplicación en PC, comunicándose con el primero mediante UART y con la segunda mediante Bluetooth.

Este módulo fue elegido por su conectividad Bluetooth integrada, que permitió implementar dicha comunicación sin hardware adicional, junto con su bajo costo y amplia disponibilidad.

El ESP32 es un SoC (*System on Chip*) de Espressif Systems [12], con doble núcleo Xtensa LX6 a 240 MHz y conectividad Wi-Fi 802.11b/g/n y Bluetooth 4.2, con soporte para Bluetooth Classic y BLE (*Bluetooth Low Energy*). En este trabajo se utilizó exclusivamente Bluetooth Classic para la transmisión de datos en formato JSON (*JavaScript Object Notation*). En la figura 2.2 se muestra el módulo, donde se distingue la antena Bluetooth integrada en el extremo superior.



FIGURA 2.2. Módulo ESP32².

2.1.3. Celda de carga y módulo HX711

Para medir el peso del alimento en el plato se utilizó una celda de carga tipo galga extensiométrica combinada con el módulo amplificador y convertor analógico-digital HX711. La celda de carga convierte la fuerza aplicada en una variación de resistencia eléctrica de muy baja amplitud. Dado que el microcontrolador no puede procesar señales analógicas de esa magnitud directamente, se incorporó el módulo HX711 que amplifica y digitaliza dicha señal con una resolución de 24 bits. En la figura 2.3 se ilustran la celda de carga en la parte superior y el módulo HX711 con sus terminales de conexión en la parte inferior.

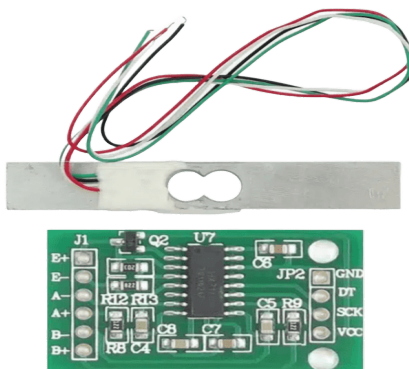


FIGURA 2.3. Celda de carga y módulo HX711³.

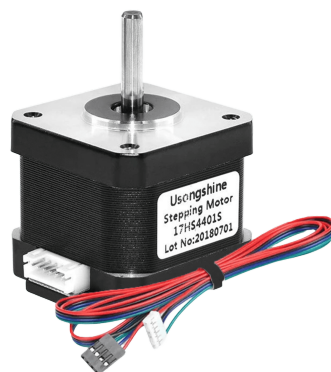
²https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf.

³https://cdn.sparkfun.com/assets/b/f/5/a/e/hx711F_EN.pdf.

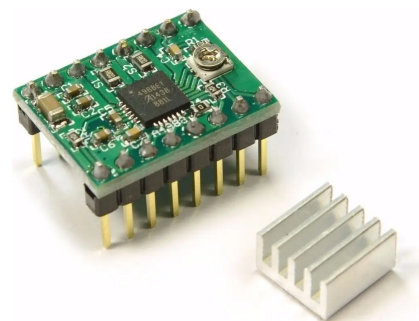
Para la comunicación con el microcontrolador, el HX711 utiliza una interfaz de dos cables (CLK y DATA) y opera con una tensión de alimentación de 5 V o 3,3 V. Su tasa de muestreo configurable es de 10 o 80 muestras por segundo, lo que resulta suficiente para la aplicación de monitoreo de consumo en intervalos regulares.

2.1.4. Motor paso a paso y driver A4988

El mecanismo de dispensado de alimento fue accionado por un motor paso a paso NEMA 17 [13], seleccionado por su precisión de movimiento y su par motor. Para su control se utilizó el driver A4988 de Pololu [14], un módulo compacto que permite gestionar motores paso a paso bipolares con corrientes de hasta 2 A por fase. En la figura 2.4 se muestran el motor NEMA 17 (figura 2.4a) y el driver A4988 junto con su disipador térmico (figura 2.4b).



(A) Motor paso a paso NEMA 17.



(B) Driver A4988.

FIGURA 2.4. Motor paso a paso NEMA 17 y driver A4988⁴.

El driver A4988 acepta señales de dirección (DIR) y paso (STEP) provenientes del microcontrolador y permite ajustar la resolución de movimiento mediante micropasos. Su tensión de alimentación lógica es de 3,3 V a 5,5 V, mientras que la tensión de potencia para el control del motor puede ser de hasta 35 V.

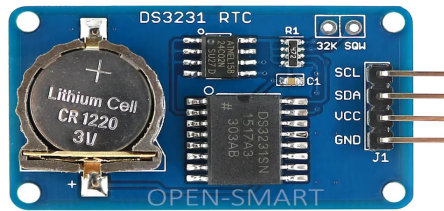
2.1.5. Módulo RTC DS3231

Para la gestión autónoma de los horarios de dispensado se incorporó un módulo de reloj de tiempo real (RTC) basado en el integrado DS3231 [15] de Maxim Integrated.

Este módulo se incorporó debido a su alta precisión y a su batería de respaldo, que permite conservar la hora y los horarios programados ante cortes de energía sin depender de una conexión a Internet.

El DS3231 proporciona información de fecha y hora con una precisión de ± 2 ppm, equivalente a aproximadamente 1 minuto de desviación por año, y se comunica con el microcontrolador mediante el protocolo I²C. La figura 2.5 muestra el módulo, que incluye una batería de litio tipo botón CR1220 como respaldo.

⁴<https://www.pololu.com/product/1182>.

FIGURA 2.5. Módulo RTC DS3231⁵.

2.2. Software y entorno de desarrollo

En esta sección se describen las herramientas de software y los entornos de desarrollo utilizados para la implementación del sistema, tanto para el firmware del microcontrolador como para el módulo inalámbrico y la aplicación de escritorio.

2.2.1. STM32CubeIDE

STM32CubeIDE [16] es un entorno de desarrollo integrado (IDE) de código abierto utilizado para la programación del firmware del microcontrolador STM32F446RE. Basado en Eclipse, permite configurar periféricos, generar código inicial de forma automática, compilar y depurar en la misma plataforma.

2.2.2. FreeRTOS

FreeRTOS es un sistema operativo de tiempo real (RTOS) de código abierto ampliamente utilizado en sistemas embebidos. Provee mecanismos de planificación de tareas, semáforos, colas de mensajes y temporizadores de software que facilitan la gestión de múltiples procesos concurrentes. En este trabajo, FreeRTOS se integró como middleware sobre la capa de abstracción de hardware (HAL) del STM32 para gestionar las tareas de lectura de sensores, control del motor, comunicación UART y manejo del pulsador.

2.2.3. Doxygen

Doxygen [17] es una herramienta de código abierto para la generación automática de documentación a partir de comentarios estructurados en el código fuente. En este trabajo se utilizó para documentar el firmware del STM32 y obtener una referencia técnica en formato HTML (*HyperText Markup Language*) que describe las funciones, parámetros y estructuras de datos implementadas.

2.2.4. PlatformIO

Para el desarrollo del firmware del módulo ESP32 se utilizó PlatformIO [18], una extensión del editor Visual Studio Code que provee soporte para múltiples plataformas de desarrollo embebido. Este entorno simplifica el acceso a las bibliotecas de comunicación Bluetooth Classic (SPP, *Serial Port Profile*).

⁵<https://www.analog.com/en/products/ds3231.html>.

2.2.5. Flutter

Flutter [19] es un framework de código abierto desarrollado por Google para la creación de aplicaciones multiplataforma a partir de una única base de código. En este trabajo se utilizó para desarrollar la aplicación de escritorio que corre en Windows y se comunica con el módulo ESP32 mediante Bluetooth Classic. Esta aplicación permite al usuario visualizar el historial de consumo y configurar los horarios de dispensado.

2.2.6. Lenguaje de programación C

El firmware del microcontrolador STM32 se desarrolló íntegramente en lenguaje C, un lenguaje de propósito general que permite el control directo del hardware y la gestión eficiente de los recursos del sistema. Su portabilidad y compatibilidad con las bibliotecas HAL de STM32 y con FreeRTOS lo convirtieron en la opción más adecuada para este tipo de desarrollo.

2.3. Protocolos de comunicación

En esta sección se describen los tres protocolos de comunicación empleados para la transferencia de datos entre los distintos módulos del sistema. La elección de cada protocolo respondió a los requisitos de interfaz de cada módulo y a decisiones de diseño del sistema.

2.3.1. Protocolo I2C

Este protocolo consiste en un bus serie síncrono, bidireccional y semidúplex, organizado bajo una arquitectura maestro-esclavo. Utiliza dos líneas eléctricas: SCL para la señal de reloj y SDA para la transferencia de datos. La comunicación se basa en tramas que incluyen la dirección de 7 bits del dispositivo, un bit de lectura/escritura y un mecanismo de confirmación mediante señales ACK (*Acknowledgement*) y NACK (*Not Acknowledgement*) [10].

2.3.2. Protocolo UART

El protocolo UART se utilizó para la comunicación serie entre el microcontrolador STM32 y el módulo ESP32. Este protocolo es asíncrono y utiliza dos líneas físicas: una de transmisión (TX) y otra de recepción (RX). Los datos se transmiten en tramas compuestas por un bit de inicio, los bits de datos (generalmente 8) y un bit de parada. En este sistema se configuraron ambos extremos con una velocidad de 115 200 baudios, sin paridad y con un bit de parada [9].

2.3.3. Bluetooth Classic

El módulo ESP32 gestionó la comunicación inalámbrica con la aplicación en PC mediante Bluetooth Classic bajo el perfil SPP (*Serial Port Profile*). Este perfil emula una comunicación serie sobre el canal Bluetooth, lo que facilita la integración con el microcontrolador, ya que los datos recibidos por UART desde el STM32 fueron reenviados por Bluetooth, y viceversa. Los datos se transmiten en formato JSON, lo que permite estructurar la información de manera legible y extensible [20].

Capítulo 3

Diseño e implementación

En este capítulo se describen el diseño e implementación del sistema desarrollado, tanto del dispositivo en general como los diferentes módulos de firmware que lo componen. También se presenta la interfaz de usuario y el protocolo de comunicación empleado entre el microcontrolador y la aplicación de escritorio.

3.1. Arquitectura general del sistema

El sistema desarrollado constituye un dispensador automático de alimento seco para gatos, capaz de operar de forma autónoma sin dependencia de una conexión a Internet. El dispositivo gestiona la entrega de alimento en horarios programados y monitorea el peso del tanque de almacenamiento y del plato de consumo. Esta información se comunica al usuario a través de una aplicación de escritorio mediante Bluetooth.

El componente central del sistema es el microcontrolador STM32F446RE, que actúa como unidad de procesamiento principal. Este componente coordina la lectura de los sensores de pesaje, controla el motor de dispensado y administra la comunicación con los módulos periféricos. La figura 3.1 presenta el diagrama en bloques del sistema, donde se distinguen cuatro bloques funcionales principales: sensado, control, actuación y comunicación con el usuario.

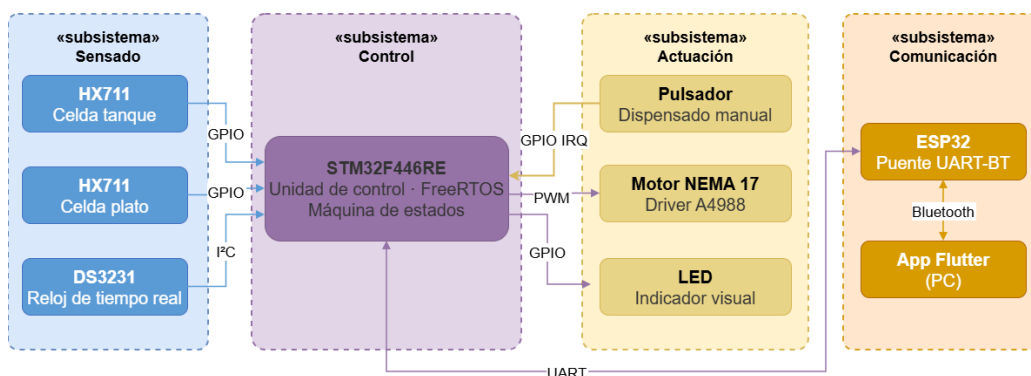


FIGURA 3.1. Diagrama en bloques del sistema.

3.1.1. Bloque de sensado

El bloque de sensado comprende dos módulos HX711 conectados a sus respectivas celdas de carga: uno ubicado bajo el tanque de almacenamiento y otro bajo el plato. Cada módulo entrega al microcontrolador una lectura digital de 24 bits proporcional al peso ejercido sobre la celda correspondiente, lo que permite estimar tanto la cantidad de alimento disponible como la porción efectivamente consumida por el animal.

A este bloque se suma el módulo DS3231, que provee la referencia temporal necesaria para la gestión autónoma de los horarios de dispensado. La disponibilidad de esta referencia de tiempo independiente de una conexión a Internet permite que el sistema mantenga la programación de horarios aun ante cortes de energía o de conectividad.

3.1.2. Bloque de control

El bloque de control está compuesto por el microcontrolador STM32F446RE, que concentra toda la lógica del sistema. Este componente recibe la información provista por el bloque de sensado y, en función de ella y de los comandos recibidos desde la aplicación, determina las acciones a ejecutar por el bloque de actuación.

La información ingresa al microcontrolador a través de distintas interfaces según el periférico: las lecturas de peso de los módulos HX711 mediante señales digitales GPIO, la referencia temporal del DS3231 a través del bus I²C, los comandos del usuario por la interfaz UART con el módulo de comunicación y el pulsador manual mediante una interrupción externa. A partir de estas entradas, el microcontrolador convierte los datos crudos en información útil: las lecturas de los HX711 se transforman en valores de peso en gramos y la hora del DS3231 se compara con los horarios programados para decidir cuándo activar el dispensado. El detalle de las interfaces y la asignación de pines se presenta en la sección [3.2.2](#).

3.1.3. Bloque de actuación

El bloque de actuación está compuesto por el motor paso a paso NEMA 17 con su driver A4988, el pulsador de dispensado manual y el LED de estado del sistema. El motor ejecuta el dispensado de alimento según las órdenes del bloque de control, mientras que el LED de estado y el pulsador permiten al usuario interactuar directamente con el dispositivo sin depender de la aplicación de escritorio, ya sea para visualizar el estado del sistema o para ejecutar un dispensado manual.

3.1.4. Bloque de comunicación con el usuario

El bloque de comunicación con el usuario está compuesto por el módulo ESP32, que opera como puente entre el microcontrolador y la aplicación de escritorio, a través de Bluetooth Classic. Este bloque es responsable de transmitir la telemetría del sistema, notificar los eventos de dispensado y recibir los comandos enviados por el usuario desde la aplicación.

3.1.5. Interacción entre hardware y software

La figura 3.2 presenta el diagrama de capas del sistema, donde se observa la relación entre los componentes de hardware descritos y los módulos de firmware encargados de gestionarlos. El firmware se organiza en torno a dos tareas concurrentes de FreeRTOS: una dedicada al control del dispensado, que interactúa directamente con los módulos HX711, DS3231, el motor NEMA 17 y el pulsador; y otra dedicada a la comunicación, que gestiona el intercambio de mensajes con la aplicación de escritorio a través del módulo ESP32. Ambas tareas se sincronizan mediante una cola de mensajes, lo que permite que el procesamiento de comandos externos no interfiera con el ciclo de control del hardware.

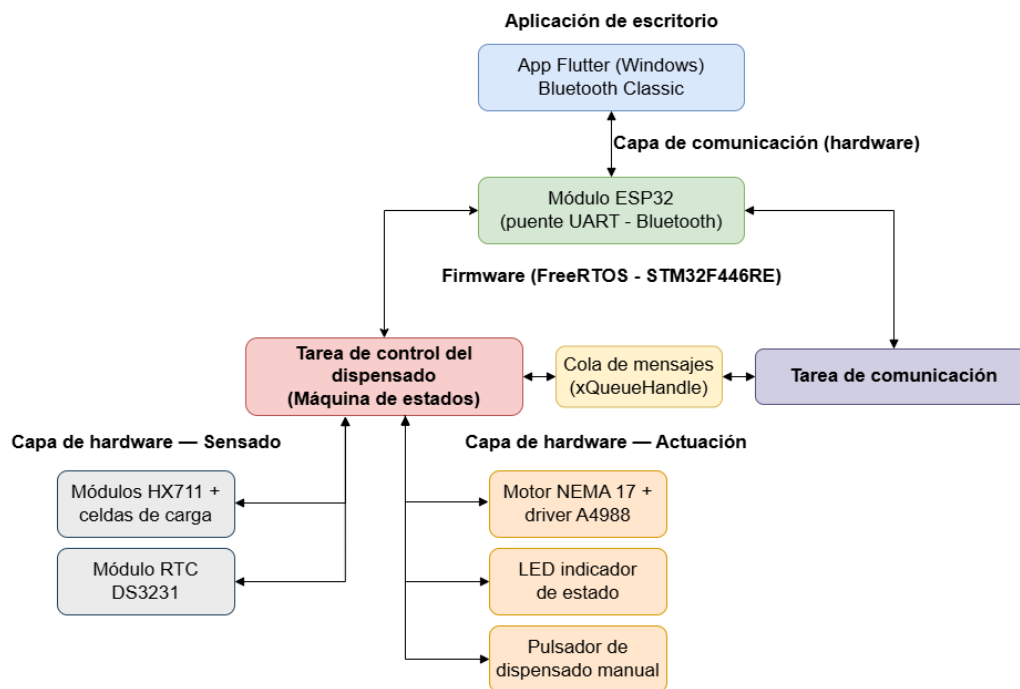


FIGURA 3.2. Diagrama de capas hardware-software del sistema.

3.2. Arquitectura de hardware

En esta sección se describe la interconexión entre los componentes electrónicos que conforman el sistema, junto con el análisis de potencia necesario para garantizar su correcto funcionamiento.

3.2.1. Gestión de alimentación

En primera instancia, se resolvió el requerimiento de alimentación del sistema. A diferencia de soluciones portátiles basadas en batería, el presente trabajo utiliza una fuente de alimentación de 12 V de corriente alterna como fuente de energía principal. Esta decisión se fundamenta en el elevado consumo del motor paso a paso NEMA 17 con su driver A4988, que en condiciones de máxima carga puede demandar hasta 2 A, lo que hace inviable el uso de una batería para una operación continua y prolongada.

Dado que la totalidad de los componentes del sistema opera a una tensión nominal de 5 V, se incorporó un regulador de voltaje reductor (step-down) basado en el integrado LM2596S [21], capaz de aceptar tensiones de entrada de hasta 35 V y entregar una corriente máxima de 3 A con tensión de salida ajustable.

Este integrado se seleccionó frente a otras alternativas de regulación debido a su mayor eficiencia energética. Al operar como convertidor conmutado, disipa significativamente menos potencia en forma de calor que un regulador lineal equivalente. Esa menor disipación resulta especialmente relevante dada la diferencia entre la tensión de entrada (12 V) y la tensión de salida requerida (5 V). De esta manera, se evita la necesidad de disipadores de gran tamaño y se reduce el riesgo de sobrecalentamiento durante la operación continua del sistema.

La salida del módulo se ajustó mediante su potenciómetro integrado hasta alcanzar los 5 V requeridos por los componentes del sistema, valor que se verificó con un multímetro antes de conectar el resto de los módulos. La figura 3.3 ilustra su conexión con el microcontrolador.

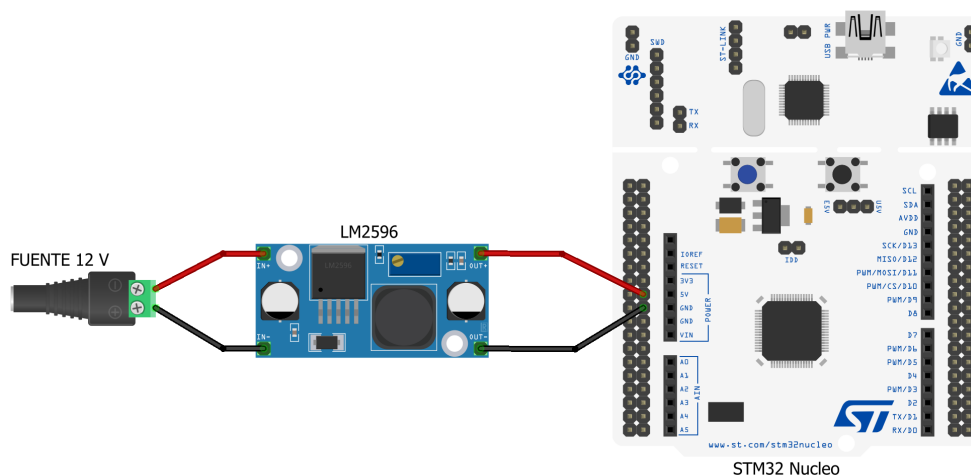


FIGURA 3.3. Interconexión entre la fuente de alimentación y el microcontrolador.

3.2.2. Interconexión de componentes

Una vez presentados los componentes del sistema en el capítulo anterior, se describe a continuación cómo se interconectan entre sí. Todos los módulos operan a una tensión nominal de 5 V, por lo que comparten la misma fuente de alimentación.

El microcontrolador STM32F446RE se comunica con los dos módulos HX711 mediante señales digitales GPIO (*General Purpose Input/Output*). Cada módulo HX711 emplea una interfaz de dos cables: uno de reloj (CLK) y uno de datos (DATA), conectados a pines GPIO del microcontrolador configurados como salida y entrada respectivamente.

El módulo DS3231 se conecta al microcontrolador a través del bus I²C, por medio de las líneas SCL y SDA del periférico I2C1, configurado a 400 kHz.

El módulo ESP32 se conecta mediante el periférico UART4, configurado a 115 200 baudios con formato 8N1.

El pulsador de dispensado manual se conecta a un pin configurado como entrada con interrupción externa (EXTI), con un tiempo de antirrebote de 200 ms implementado por software.

El LED de estado se conecta a una salida GPIO de propósito general. La figura 3.4 muestra el esquema de estas conexiones.

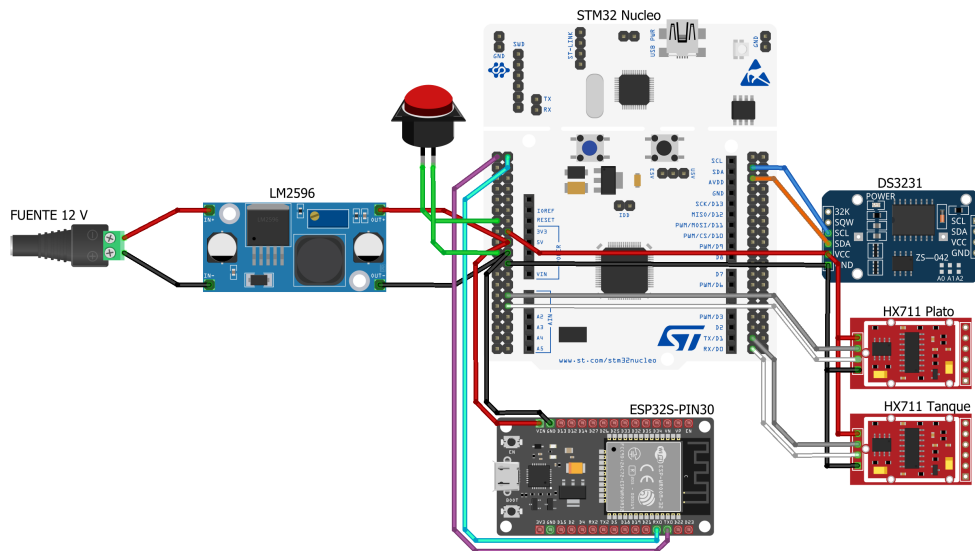


FIGURA 3.4. Esquema de conexiones del microcontrolador.

El motor paso a paso NEMA 17 se controla a través del driver A4988, que recibe tres señales digitales provenientes del microcontrolador. La señal STEP, generada por el canal 3 del temporizador TIM2 en modo PWM [22] sobre el pin PB10, que determina la velocidad y el número de pasos ejecutados por el motor. La señal DIR define el sentido de giro, mientras que la señal ENABLE habilita o deshabilita el driver. Las señales DIR y ENABLE se manejan como salidas GPIO simples. La figura 3.5 ilustra el esquema de conexión entre el microcontrolador, el driver y el motor.

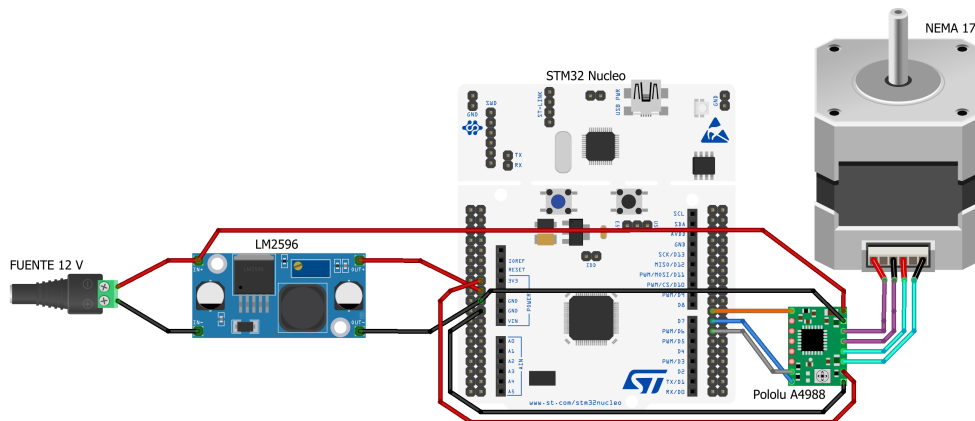


FIGURA 3.5. Interconexión entre el microcontrolador, el driver A4988 y el motor NEMA 17.

La tabla 3.1 resume la asignación de pines del microcontrolador STM32F446RE a cada uno de los periféricos descritos en esta sección.

TABLA 3.1. Asignación de pines del microcontrolador STM32F446RE.

Periférico	Pin	Función	Configuración
HX711 (tanque) — DOUT	PC0	Lectura de dato	GPIO entrada
HX711 (tanque) — SCK	PC1	Reloj de muestreo	GPIO salida
HX711 (plato) — DOUT	PC2	Lectura de dato	GPIO entrada
HX711 (plato) — SCK	PC3	Reloj de muestreo	GPIO salida
DS3231 — SCL	PB8	Reloj I ² C	I ² C1, 400 kHz
DS3231 — SDA	PB9	Datos I ² C	I ² C1, 400 kHz
ESP32 — TX	PA0	Transmisión UART	UART4, 115 200 baudios, 8N1
ESP32 — RX	PA1	Recepción UART	UART4, 115 200 baudios, 8N1
Pulsador manual	PC13	Dispensado manual	EXTI, antirrebote 200 ms
LED de estado	PA5	Indicación de estado	GPIO salida
Motor — STEP	PB10	Señal de pasos	TIM2 canal 3, PWM
Motor — DIR	PA6	Sentido de giro	GPIO salida
Motor — ENABLE	PA7	Habilitación del driver	GPIO salida (activo en bajo)

3.2.3. Análisis de potencia

El prototipo desarrollado requiere una fuente de alimentación capaz de proveer la corriente suficiente para todos los componentes del sistema de forma simultánea. La tabla 3.2 resume el consumo máximo de cada componente según las hojas de datos de sus fabricantes.

TABLA 3.2. Consumo máximo de cada componente del sistema.

Componente	Tensión (V)	Corriente máxima (mA)
STM32F446RE	5	36
HX711 (×2)	5	3
DS3231	5	1
Driver A4988	5	10
Motor NEMA 17	12	1700
ESP32	5	130
LED de estado	5	20

Del análisis de la tabla se concluye que el componente de mayor consumo es el motor paso a paso NEMA 17, que opera a 12 V y puede demandar hasta 1,7 A por fase en condiciones de máxima carga. El resto de los componentes opera a 5 V con un consumo total aproximado de 200 mA. Estos valores justifican la elección de una fuente de alimentación de 12 V y el módulo regulador LM2596S, cuya corriente máxima de salida de 3 A provee el margen necesario para una operación confiable del sistema.

3.3. Diseño del firmware

En esta sección se describe la organización del firmware desarrollado para el microcontrolador STM32F446RE y la lógica de control que gobierna el comportamiento del sistema.

3.3.1. Organización del firmware

Para implementar la lógica del sistema, se desarrollaron módulos de software de forma independiente, con el propósito de que su desarrollo fuera autónomo y para facilitar su integración. La figura 3.6 muestra el diagrama en bloques que representa los subsistemas que conforman el firmware.

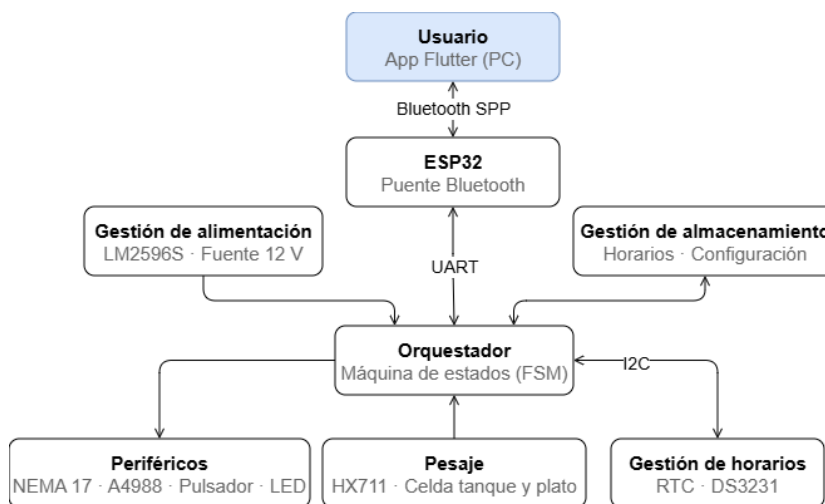


FIGURA 3.6. Diagrama en bloques de los subsistemas de software.

El módulo central del sistema es el orquestador, cuya función es inicializar todos los periféricos necesarios y luego gestionar la lógica de control a través de una máquina de estados. Para ello se utilizó FreeRTOS, con dos tareas principales: una encargada del control del dispensado y otra dedicada a la comunicación con el usuario.

La elección de un sistema operativo de tiempo real en lugar de un esquema de bucle principal único respondió a la necesidad de atender dos actividades de naturaleza distinta sin que una bloqueara a la otra. El control del dispensado exige un ciclo determinista que consulte los sensores y el reloj de tiempo real, mientras que la comunicación permanece a la espera de mensajes que llegan de forma asíncrona desde la aplicación. FreeRTOS permitió asignar cada actividad a una tarea independiente con su propia prioridad y delegar en el planificador la conmutación entre ambas. Esta separación simplificó el diseño y evitó las máquinas de estado anidadas que habría requerido una solución basada en un único bucle con sondeo permanente.

3.3.2. Módulo orquestador

El módulo orquestador gestiona el ciclo completo de operación del dispensador mediante una máquina de estados finita.

El estado `ST_INIT` corresponde a la inicialización del sistema: se configuran los periféricos, se establece la comunicación con los módulos HX711 y DS3231, y se ejecuta la tara automática de las celdas de carga. Si la inicialización concluye de forma exitosa, el sistema transita al estado `ST_IDLE`. Ante un error irreparable, transita al estado `ST_ERROR`, donde el LED permanece encendido y el sistema realiza un reinicio automático tras 10 s.

En el estado `ST_IDLE`, el sistema aguarda eventos: lee periódicamente el reloj de tiempo real, transmite la telemetría hacia la aplicación y evalúa la ocurrencia de un horario programado o de una pulsación manual. Cuando se detecta el evento `EVT_BUTTON` o `EVT_SCHEDULED_FEED`, el sistema transita al estado `ST_DISPENSING`, donde activa el motor, notifica al usuario y aguarda la confirmación del ciclo. Al completarse el dispensado, retorna a `ST_IDLE`. La figura 3.7 presenta el diagrama de estados del sistema.

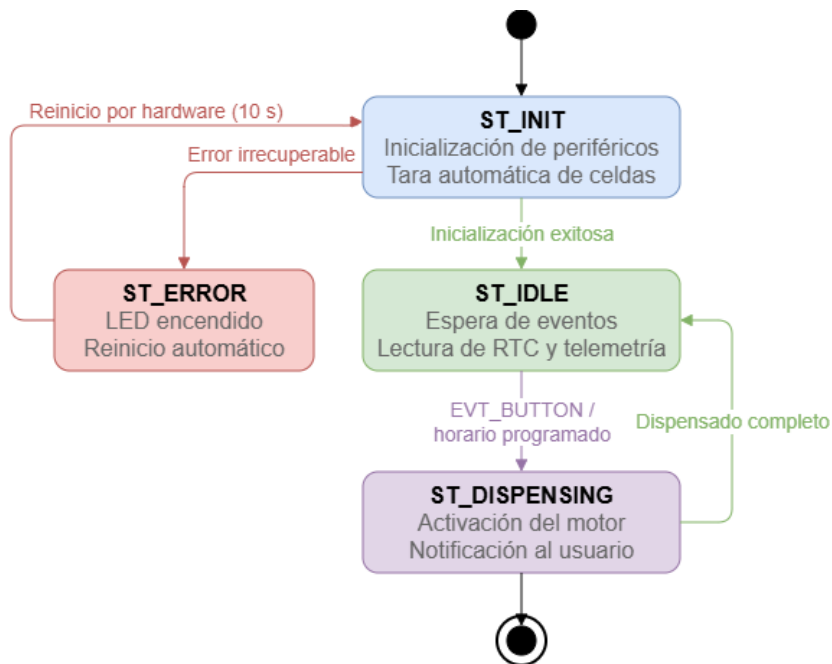


FIGURA 3.7. Diagrama de estados del dispensador.

La figura 3.8 presenta el diagrama de secuencia del ciclo de dispensado, que puede iniciarse por dos vías: mediante el comando `dispense` recibido desde la aplicación o de forma autónoma. En el primer caso, `CommTask` traduce el comando en el evento `EVT_BUTTON` y lo publica en la cola de eventos. En el segundo, `SMTask` detecta durante el polling periódico de `ST_IDLE` que la hora provista por el DS3231 coincide con un horario programado y genera internamente el evento `EVT_SCHEDULED`.

En ambos casos, el sistema transita a `ST_DISPENSING`, donde se registra el evento en el log y se invoca `Motor_Dispense()`. Esta función configura el temporizador `TIM2` para generar la secuencia de pulsos `STEP` de forma autónoma por

hardware; el conteo de pasos se realiza en la rutina de interrupción del temporizador, sin bloquear a SMTask. Mientras el motor opera, SMTask permanece en un ciclo de espera de 100 ms consultando `Motor_IsDone()`. Al completarse la secuencia de pasos, el sistema retorna a `ST_IDLE` y notifica el evento a la aplicación mediante `Comm_SendFeedingDone()`, que incluye la cantidad dispensada, el origen del evento (manual o programado) y la hora correspondiente.

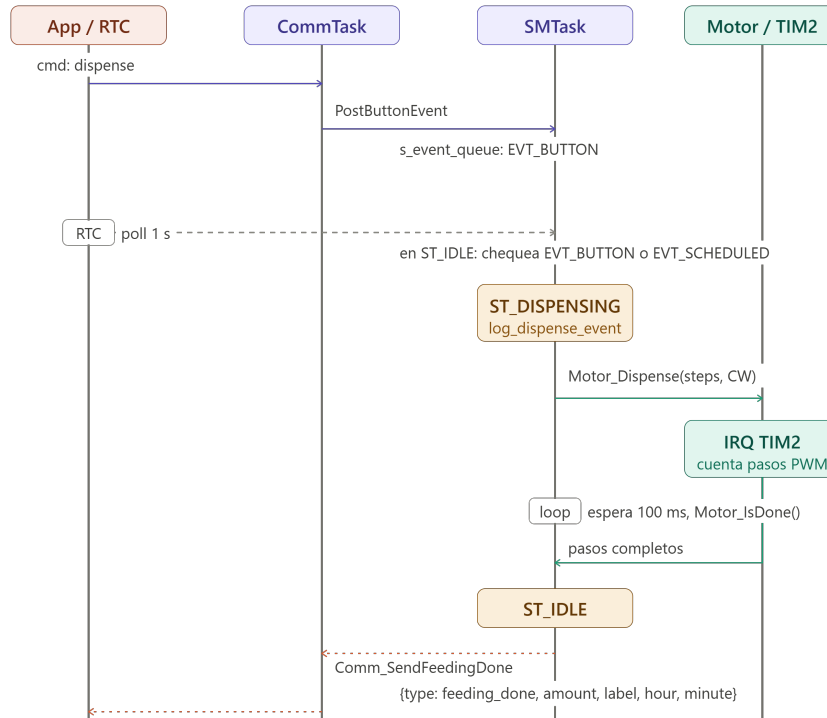


FIGURA 3.8. Diagrama de secuencia del ciclo de dispensado.

El estado `ST_ERROR` se alcanza exclusivamente si la inicialización del módulo DS3231 falla de forma irre recuperable. Durante `ST_INIT`, el sistema intenta inicializar y configurar la hora del RTC hasta `BOOT_MAX_TRIES` veces (tres intentos), con una espera de 200 ms entre cada uno. Si el RTC no responde tras agotar los reintentos, o si la configuración inicial de fecha y hora falla, el sistema transita a `ST_ERROR`.

En este estado, el LED de estado permanece encendido de forma continua como indicación visual de la falla. El sistema permanece en espera durante 10 s y luego ejecuta `NVIC_SystemReset()` para reiniciar por completo el microcontrolador. Al reiniciar el microcontrolador, se ejecuta nuevamente la secuencia completa de inicialización, lo que permite recuperarse de fallas transitorias de comunicación I²C con el RTC sin intervención manual.

3.3.3. Módulo de periféricos

El módulo de periféricos agrupa los componentes de actuación y señalización del sistema: el motor paso a paso NEMA 17, el pulsador de dispensado manual y el LED de estado.

El control del motor se implementó mediante el driver A4988. El giro del eje del motor se logra mediante una secuencia de pulsos generados sobre la señal `STEP`, donde cada pulso corresponde a un paso del motor. Esta señal se genera por el canal 3 del temporizador `TIM2`, configurado en modo PWM para producir el tren de pulsos de forma autónoma por hardware. La frecuencia de dicho tren determina la velocidad de rotación del motor, mientras que la cantidad de pulsos define el ángulo total recorrido. Las señales `DIR` y `ENABLE` controlan el sentido de giro y la habilitación del driver respectivamente. Ambas se implementaron como salidas GPIO de propósito general. La cantidad de pasos ejecutados en cada ciclo de dispensado determina la porción de alimento entregada.

El pulsador de dispensado manual se conecta a un pin configurado con interrupción externa (`EXTI`). La rutina de servicio de interrupción implementa un mecanismo de antirrebote de 200 ms por software y publica un evento en la cola de eventos de FreeRTOS para notificar a la tarea de control del dispensado.

El LED de estado refleja el estado activo de la máquina de estados: permanece encendido de forma continua en `ST_ERROR`, parpadea a 1 Hz durante `ST_IDLE` y se apaga en `ST_DISPENSING`.

3.3.4. Módulo de pesaje

El módulo de pesaje gestiona la inicialización y la lectura periódica de los dos módulos HX711 conectados a las celdas de carga del tanque y del plato.

Durante el estado `ST_INIT`, el módulo ejecuta una tara automática que establece el valor de referencia de reposo de cada celda. A partir de ese punto, las lecturas de peso se obtienen mediante la aplicación de un factor de calibración sobre la diferencia entre el valor digital de 24 bits provisto por el HX711 y dicho offset de tara, y el resultado se expresa en gramos.

El factor de calibración se determinó experimentalmente. Se pesó un objeto de referencia en una balanza doméstica para obtener su masa real y luego se contrastó este valor con la lectura cruda entregada por el HX711 para el mismo objeto, lo que permitió calcular la relación entre ambas magnitudes. Para reducir el ruido en la señal, cada lectura resulta del promedio de 8 muestras consecutivas, obtenidas en una misma operación de bloque previa al cálculo del valor en gramos.

3.3.5. Módulo de gestión de horarios

El módulo de gestión de horarios compara periódicamente la hora provista por el módulo DS3231 con la lista de horarios de dispensado almacenados. En cada ciclo de la tarea de control, durante el estado `ST_IDLE`, se consultan las horas y los minutos del reloj de tiempo real y se verifica si alguno de los horarios activos coincide con el instante actual. Cuando se detecta una coincidencia, se genera el evento `EVT_SCHEDULED_FEED`, que desencadena la transición al estado `ST_DISPENSING`. Como la comparación se realiza a nivel de hora y minuto, la

coincidencia se mantiene durante todo ese minuto. Para evitar que en un mismo horario se disparen varios eventos, un mecanismo de inhibición recuerda el último horario activado y no vuelve a dispararlo hasta el minuto siguiente.

3.3.6. Módulo de gestión de almacenamiento

El módulo de gestión de almacenamiento tiene como función conservar la configuración del sistema a través de los ciclos de encendido y apagado. Los horarios de dispensado se almacenan en la memoria flash interna del microcontrolador STM32F446RE, lo que permite recuperarlos ante un eventual reinicio del sistema.

Se optó por la memoria flash interna del microcontrolador en lugar de una memoria externa dedicada, dado que el volumen de configuración a conservar es reducido y no justifica el costo ni el espacio de un componente adicional. Esta decisión aprovechó un recurso ya disponible en el STM32F446RE y simplificó el diseño del circuito, sin líneas de comunicación ni componentes extra.

El sistema admite hasta diez horarios programados. Cada entrada incluye la hora (0–23), los minutos (0–59), un indicador de habilitación y una etiqueta de texto identificatoria. Al iniciarse el sistema, los datos se cargan desde la flash hacia una estructura en memoria RAM, donde son consultados por el módulo de gestión de horarios durante la operación normal.

La actualización de los horarios se origina en la aplicación de escritorio, que envía el comando `set_schedules` al sistema a través del protocolo de comunicación descrito en la sección 3.4.1. Al recibir este comando, el módulo reemplaza el contenido previo en flash con los nuevos valores.

3.4. Comunicaciones e interfaces

En esta sección se describe la implementación del protocolo de comunicación entre el microcontrolador y la aplicación de escritorio, así como las funcionalidades y la estructura de navegación de la interfaz de usuario.

3.4.1. Módulo de comunicación

El módulo de comunicación gestiona el intercambio de información entre el microcontrolador y la aplicación de escritorio a través del módulo ESP32. La comunicación es bidireccional y se estructura en dos tipos de mensajes: los mensajes salientes, que el sistema envía hacia la aplicación, y los comandos entrantes, que la aplicación envía hacia el sistema. Todos los mensajes se estructuran en formato JSON [23], lo que permite una integración sencilla con la aplicación de escritorio.

Se adoptó el formato JSON en lugar de un protocolo binario propio por su legibilidad y por la disponibilidad de bibliotecas de análisis sintáctico tanto en el firmware como en la aplicación de escritorio. Esta característica facilitó la depuración durante el desarrollo, ya que los mensajes resultan interpretables de forma directa en un monitor serie. El reducido volumen de datos intercambiado torna despreciable la diferencia de eficiencia frente a una codificación binaria. La figura 3.9 ilustra la arquitectura de comunicación del sistema junto con los mensajes intercambiados.

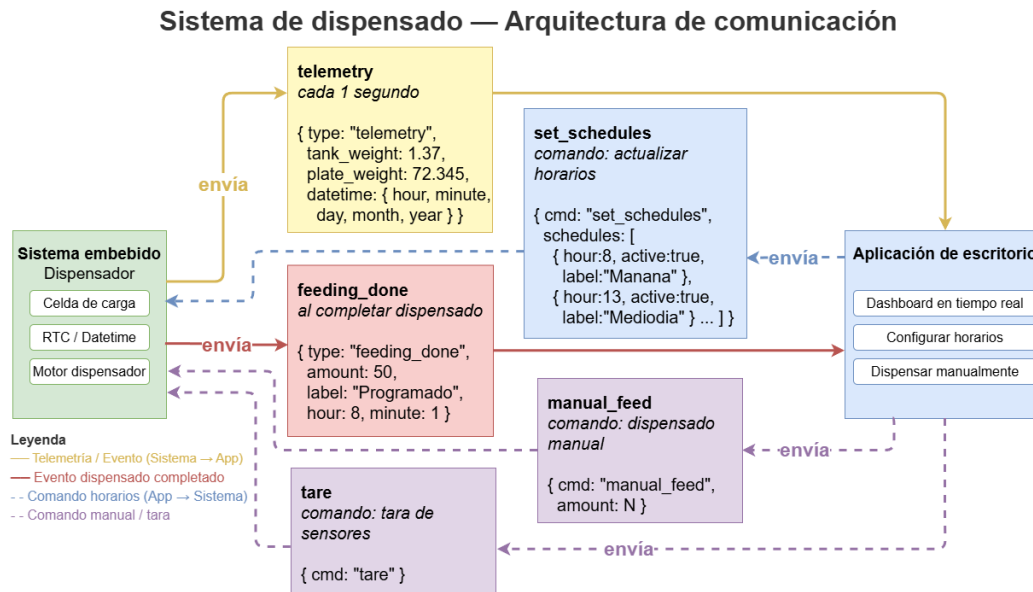


FIGURA 3.9. Arquitectura de comunicación del sistema.

El sistema transmite dos tipos de mensajes hacia la aplicación. El primero es un mensaje de telemetría transmitido cada 1 segundo que incluye el peso del tanque de almacenamiento, el peso del plato y la fecha y hora provistas por el módulo DS3231. Al completarse un ciclo de dispensado, el sistema emite un mensaje de evento con la cantidad dispensada, la etiqueta del tipo de dispensado y la hora en que ocurrió. Entre los comandos entrantes, la aplicación puede solicitar un dispensado manual mediante `manual_feed`, ejecutar la tara de las celdas de carga mediante `tare`, o actualizar los horarios programados mediante `set_schedules`. La tabla 3.3 detalla la estructura de los mensajes intercambiados entre el sistema y la aplicación, con los campos que componen cada uno.

TABLA 3.3. Estructura de los mensajes del protocolo de comunicación.

Mensaje	Dirección	Campos principales
Telemetría	Sistema → App	Peso del tanque, peso del plato, fecha y hora
Dispensado completado	Sistema → App	Cantidad dispensada, tipo de dispensado, hora del evento
Confirmación de tara	Sistema → App	Resultado de la tara de ambas celdas
Dispensado manual	App → Sistema	Solicitud de dispensado inmediato
Tara remota	App → Sistema	Solicitud de tara de las celdas de carga
Actualización de horarios	App → Sistema	Lista de horarios con hora, minuto y estado

3.4.2. Interfaz de usuario

La interfaz de usuario se implementó como una aplicación de escritorio desarrollada en Flutter, que corre sobre el sistema operativo Windows y se comunica con el sistema mediante Bluetooth Classic. La aplicación permite al usuario visualizar en tiempo real el peso del tanque de almacenamiento y del plato de consumo, consultar el historial de dispensados, configurar los horarios programados y ejecutar un dispensado manual o una tara de las celdas de carga.

La aplicación se organiza en cinco pantallas principales accesibles desde una barra de navegación inferior: Inicio, Horarios, Historial, Vincular y Ajustes.

La pantalla de inicio muestra el dashboard principal con los valores de peso actualizados en tiempo real y el estado de la última alimentación registrada. La pantalla de horarios permite consultar y gestionar los horarios programados. La pantalla de historial presenta el registro completo de eventos de dispensado con fecha, hora y cantidad. La pantalla de vinculación facilita el proceso de conexión Bluetooth con el dispositivo. Finalmente, la pantalla de ajustes permite configurar el perfil de la mascota y los parámetros generales de la aplicación.

La elección de Flutter como entorno de desarrollo respondió a su capacidad de generar aplicaciones de escritorio a partir de una única base de código, junto con la disponibilidad de bibliotecas para la gestión de la conexión Bluetooth y la construcción de interfaces gráficas. Esta característica permitió concentrar el esfuerzo de desarrollo en la lógica de la aplicación y en la experiencia de usuario, en lugar de la programación de bajo nivel de la interfaz.

La figura 3.10 presenta el diagrama de navegación de la aplicación con el contenido de cada pantalla.

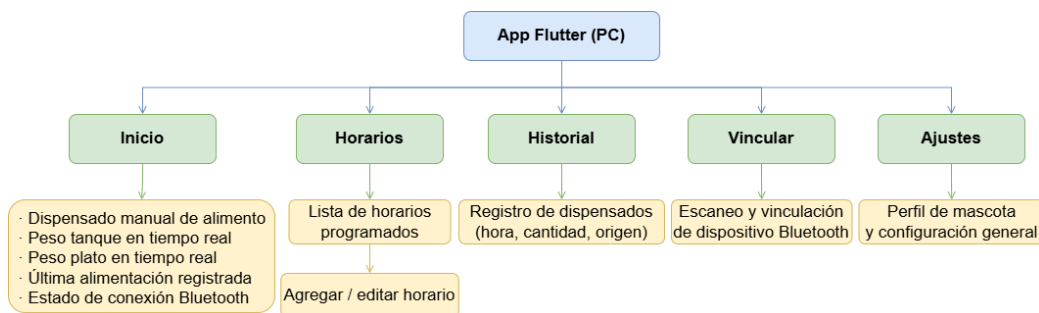


FIGURA 3.10. Diagrama de navegación de la aplicación.

3.5. Integración hardware-software

En esta sección se describe cómo los módulos de hardware y firmware interactúan durante la operación del sistema. La figura 3.11 ilustra el diagrama de integración, donde se observa la relación entre las tareas de FreeRTOS y los componentes físicos del sistema.

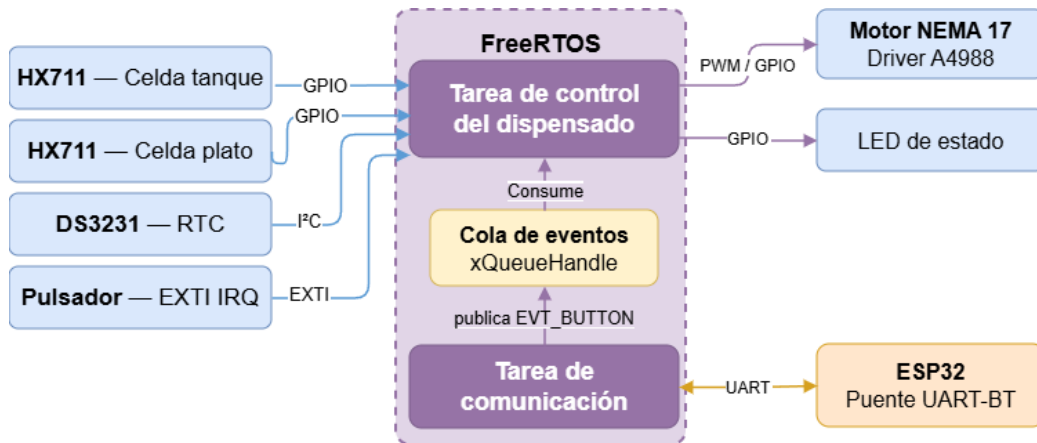


FIGURA 3.11. Diagrama de integración hardware-software.

El firmware se estructuró en torno a dos tareas concurrentes de FreeRTOS: la tarea de control del dispensado (SMTask) y la tarea de comunicación (CommTask). La primera gestiona directamente los periféricos de hardware: lee el peso de las celdas de carga a través de los módulos HX711, consulta la hora al módulo DS3231, controla el motor NEMA 17 y atiende las interrupciones del pulsador. La segunda se encarga exclusivamente del intercambio de mensajes con la aplicación de escritorio a través del módulo ESP32 por UART.

La sincronización entre ambas tareas se realiza mediante una cola de eventos de FreeRTOS (`s_event_queue`), de uso unidireccional desde CommTask hacia SMTask. Cuando CommTask recibe el comando `dispense` desde la aplicación, publica en esta cola el evento `EVT_BUTTON`, que SMTask consume durante el estado `ST_IDLE` para disparar la transición a `ST_DISPENSING`. Los comandos `tare` y `set_schedules`, en cambio, son atendidos directamente por CommTask sin involucrar a SMTask. El comando `set_schedules` actualiza una estructura interna de horarios protegida por un mutex [24] dedicado, mientras que el comando `tare` se describe en detalle más adelante en esta sección.

La notificación de eventos hacia la aplicación de escritorio, como la telemetría periódica o la confirmación de un ciclo de dispensado completo, no se realiza mediante la cola compartida, sino a través de llamadas directas a las funciones de envío del módulo de comunicación. Este esquema mantiene desacopladas ambas tareas: SMTask opera con un ciclo de polling de 1 segundo sobre la cola de eventos, mientras que CommTask permanece bloqueada esperando datos por UART y reanuda su ejecución al producirse la interrupción del periférico.

Un tercer flujo de integración, distinto del ciclo de dispensado descrito en la sección 3.3.2, es el de la tara remota de las celdas de carga. La figura 3.12 presenta el diagrama de secuencia de esta operación. Cuando CommTask recibe el comando `tare` desde la aplicación, no publica ningún evento en la cola compartida con SMTask. En cambio, accede directamente al módulo HX711 mediante el mutex que protege dicho recurso e invoca `HX711_TareAll()` para restablecer el offset de referencia de ambas celdas. A continuación, realiza una lectura de verificación sobre ambos canales para confirmar que el resultado quede próximo a cero, y notifica a la aplicación mediante `Comm_SendTareDone()`.

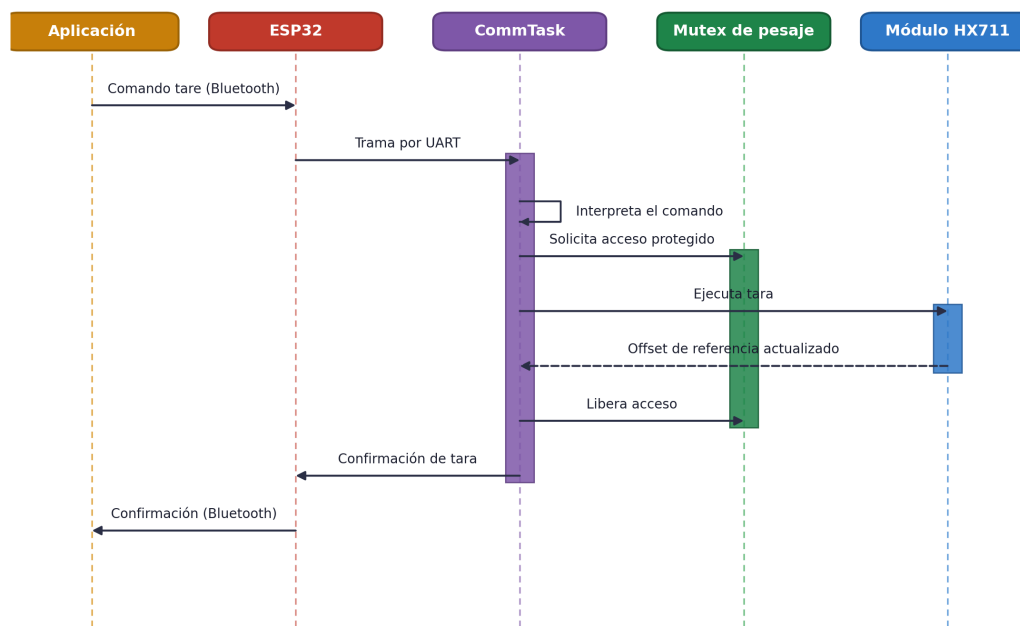


FIGURA 3.12. Diagrama de secuencia de la operación de tara remota.

La distinción entre ambos flujos responde a la naturaleza de cada operación. El dispensado modifica el estado de la máquina de estados y requiere activar el motor, por lo que exige coordinación entre las dos tareas a través de la cola de eventos. La tara, en cambio, es una operación de configuración que `CommTask` resuelve por sí sola, sin afectar el flujo normal de `ST_IDLE`.

Capítulo 4

Ensayos y resultados

En este capítulo se describen las pruebas realizadas para validar el correcto funcionamiento del sistema desarrollado. A continuación, se detallan los ensayos de hardware, de firmware y de integración, y finalmente se expone un análisis crítico de los resultados obtenidos.

4.1. Plan de ensayos

Esta sección describe la metodología de verificación adoptada, los instrumentos utilizados y los criterios de aceptación que se aplicaron para determinar el éxito de cada prueba.

4.1.1. Metodología

La estrategia de verificación se organizó en tres etapas sucesivas y dependientes entre sí. En la primera etapa se realizaron ensayos de hardware sobre cada componente electrónico de forma individual, con el objetivo de confirmar el correcto funcionamiento de las señales, los niveles de tensión y la respuesta eléctrica de cada módulo antes de incorporarlo al sistema. En la segunda etapa se ejecutaron ensayos de firmware que verificaron el comportamiento de los módulos de software: la máquina de estados, la gestión de tareas concurrentes en FreeRTOS y el protocolo de comunicación. En la tercera etapa se realizaron ensayos de integración con el prototipo completo en operación, donde se evaluaron los escenarios de uso real del sistema.

Todos los ensayos se realizaron sobre el circuito ensamblado en una placa de pruebas (protoboard) mediante cables de conexión tipo jumper, lo que simuló las condiciones del entorno de trabajo previsto.

4.1.2. Instrumentos utilizados

Los instrumentos y herramientas empleados durante los ensayos fueron los siguientes:

- Analizador lógico Saleae [25], utilizado para la observación de señales PWM, I²C y UART.
- Multímetro digital, empleado para la medición de tensiones de alimentación y verificación de continuidad en el cableado.
- Terminal serie (software), utilizada para registrar los valores transmitidos por el microcontrolador a través del puerto UART.

- Masas de referencia calibradas, empleadas para la validación del sistema de pesaje.
- Aplicación de escritorio desarrollada en Flutter, utilizada como interfaz de usuario durante los ensayos de integración.

4.1.3. Criterios de aceptación

La tabla 4.1 resume los criterios de aceptación definidos para cada prueba. Cada criterio establece la condición mínima que debe satisfacerse para considerar superado el ensayo correspondiente.

TABLA 4.1. Criterios de aceptación por módulo.

Módulo	Criterio de aceptación
Pesaje (HX711)	Variación máxima inferior a 2 g entre lecturas sucesivas sin cambio de carga.
Motor NEMA 17	Respuesta correcta en todo el rango de velocidades ensayado, sin pérdida de pasos.
RTC DS3231	Conservación de la hora tras un ciclo de apagado, con exactitud verificada durante una hora.
Comunicación	Transmisión y recepción de mensajes JSON sin pérdidas ni errores de formato.
Pulsador	Generación de exactamente un evento por accionamiento válido, sin eventos espurios por rebote.
LED de estado	Comportamiento visual conforme al estado activo de la máquina de estados en todo momento.
Integración	Ausencia de condiciones de carrera, bloqueos de tareas o pérdida de mensajes durante la operación continua.

4.2. Ensayos de hardware

En esta sección se presentan las pruebas realizadas sobre los componentes electrónicos del sistema. Para cada módulo se documentaron las condiciones de medición, las señales observadas y los resultados obtenidos.

4.2.1. Celda de carga y módulo HX711

Para verificar el correcto funcionamiento del sistema de pesaje, se conectó la celda de carga al módulo HX711 y este al microcontrolador STM32F446RE mediante las líneas CLK y DATA, según lo descrito en la sección 3.2.2. La tensión de alimentación del módulo se verificó con el multímetro y resultó de 4,98 V, dentro del rango nominal de operación.

La prueba contempló tres escenarios: una lectura sin carga sobre la celda para verificar la tara, la colocación de una masa de referencia conocida para validar la respuesta del sistema y una lectura continua durante diez minutos para detectar deriva en la señal. En la figura 4.1 se observa la salida de la terminal serie durante la lectura continua, donde el campo `tank_weight` refleja el peso registrado por la celda de carga. El valor se mantuvo estable entre lecturas sucesivas, sin variaciones que excedieran el criterio de aceptación.

```
[UART+STM32] {"type":"telemetry","tank_weight":0.05,"plate_weight":0.000,"datetime":{"hour":8,"minute":14,"second":10,"day":12,"month":4,"year":25,"weekday":1}}
[UART+STM32] {"type":"telemetry","tank_weight":0.05,"plate_weight":0.000,"datetime":{"hour":8,"minute":14,"second":12,"day":12,"month":4,"year":25,"weekday":1}}
[UART+STM32] {"type":"telemetry","tank_weight":0.05,"plate_weight":0.000,"datetime":{"hour":8,"minute":14,"second":14,"day":12,"month":4,"year":25,"weekday":1}}
[UART+STM32] {"type":"telemetry","tank_weight":0.05,"plate_weight":0.000,"datetime":{"hour":8,"minute":14,"second":16,"day":12,"month":4,"year":25,"weekday":1}}
[UART+STM32] {"type":"telemetry","tank_weight":0.05,"plate_weight":0.000,"datetime":{"hour":8,"minute":14,"second":18,"day":12,"month":4,"year":25,"weekday":1}}
```

FIGURA 4.1. Lecturas sucesivas del sistema de pesaje registradas en la terminal serie.

Tras la inicialización y la tara, el sistema registró valores estables y coherentes con la masa de referencia empleada. La variación máxima observada entre lecturas sucesivas sin cambio de carga fue inferior a 2 g, lo que satisfizo el criterio de aceptación establecido.

4.2.2. Motor paso a paso NEMA 17 y driver A4988

La verificación del mecanismo de dispensado se realizó sobre el montaje del motor NEMA 17 con el driver A4988, alimentado a 12 V. Con el analizador lógico se capturó la señal STEP generada por el canal 3 del temporizador TIM2 en modo PWM sobre el pin PB10, y se verificó que la forma de onda correspondiera a la frecuencia configurada por software. Las señales GPIO DIR y ENABLE se verificaron con el multímetro, y se confirmaron los niveles lógicos correctos en cada estado.

Se evaluaron tres condiciones: la rotación en sentido horario y antihorario mediante el cambio de estado de la señal DIR, la habilitación y deshabilitación del driver, y la variación de la frecuencia de la señal STEP para comprobar el rango de velocidades.

El motor respondió correctamente en todo el rango de velocidades ensayado. No se detectó pérdida de pasos ni calentamiento excesivo del driver en operación normal, por lo que se consideró superado el criterio de aceptación correspondiente.

4.2.3. Módulo RTC DS3231

Para evaluar el módulo de reloj de tiempo real, se estableció la comunicación I²C entre el STM32F446RE y el DS3231 mediante las líneas SCL y SDA del periférico I²C1, configurado a 400 kHz. Las señales del bus se verificaron con el analizador lógico Saleae. Los niveles lógicos y los tiempos de establecimiento se encontraron dentro de las especificaciones del protocolo.

Se comprobaron los siguientes aspectos:

- La escritura y lectura correcta de fecha y hora en los registros del dispositivo.
- La conservación de la hora tras un ciclo de apagado, con la batería de respaldo CR1220 desconectada del circuito principal.
- La exactitud de la temporización comparada con una referencia externa durante un período de observación de una hora.

La figura 4.2 presenta la captura del analizador lógico sobre el bus I²C durante una lectura del DS3231. Se observa la dirección del dispositivo (0x68) en las operaciones de escritura y lectura, junto con los registros de fecha y hora codificados en BCD. El último byte se transmite con un NAK del maestro, lo que corresponde al cierre normal de una lectura I²C y confirma el cumplimiento del protocolo.

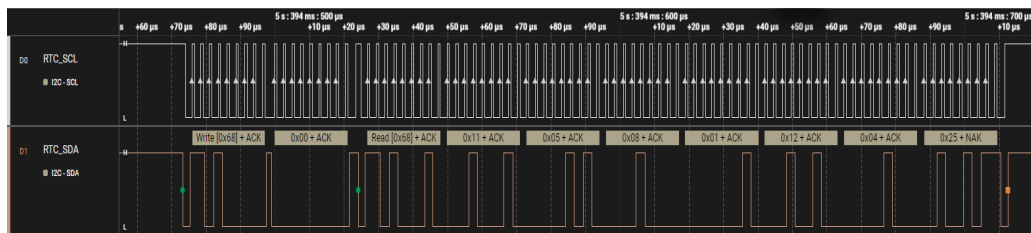


FIGURA 4.2. Captura del bus I²C entre el STM32 y el DS3231 obtenida con el analizador lógico.

Los datos de fecha y hora se leyeron correctamente en todos los ciclos. La hora se conservó tras la desconexión de la alimentación principal, lo que confirmó el correcto funcionamiento de la batería de respaldo.

4.2.4. Pulsador de dispensado manual

Para validar el pulsador, se configuró el pin de entrada del STM32F446RE con interrupción externa (EXTI). Con el analizador lógico se capturó la señal en el pin de entrada durante el accionamiento, lo que permitió caracterizar el rebote mecánico del contacto y confirmar que el tiempo de antirrebote de 200 ms implementado por software resultó suficiente para suprimirlo.

Se registraron 50 pulsaciones consecutivas en diferentes intervalos de tiempo. En todos los casos se generó exactamente un evento de interrupción por accionamiento sin eventos espurios, lo que satisfizo el criterio de aceptación definido.

4.2.5. LED de estado del sistema

La prueba del LED de estado consistió en verificar con el multímetro la tensión sobre el componente en cada estado de la máquina de estados. Se comprobó que el LED permaneciera encendido de forma continua en el estado `ST_ERROR`, parpadeara a 1 Hz en el estado `ST_IDLE` y se apagara en el estado `ST_DISPENSING`. Todos los patrones de iluminación respondieron conforme a lo especificado.

4.3. Ensayos de firmware

En esta sección se presentan las pruebas de las funciones de software: el comportamiento de la máquina de estados, la gestión de tareas concurrentes bajo FreeRTOS y el protocolo de comunicación, junto con la verificación del timing de los mensajes de telemetría.

4.3.1. Máquina de estados y módulo orquestador

Se verificó el comportamiento de la máquina de estados finita implementada en el módulo orquestador, descrita en la sección 3.3.2, mediante la ejecución de transiciones forzadas entre estados. Para cada transición se registró por la terminal serie el estado de origen, el evento disparador y el estado de destino, y se compararon los resultados con el diagrama de la figura 3.7.

La tabla 4.2 resume los casos de prueba ejecutados y los resultados obtenidos.

TABLA 4.2. Resultados de las pruebas unitarias de la máquina de estados.

Estado inicial	Evento	Estado esperado	Resultado
ST_INIT	Inicialización exitosa	ST_IDLE	OK
ST_INIT	Error irrecuperable	ST_ERROR	OK
ST_IDLE	EVT_BUTTON	ST_DISPENSING	OK
ST_IDLE	EVT_SCHEDULED_FEED	ST_DISPENSING	OK
ST_DISPENSING	Dispensado completo	ST_IDLE	OK
ST_ERROR	Reinicio tras 10 s	ST_INIT	OK

Todos los casos de prueba produjeron las transiciones esperadas. El mecanismo de inhibición que evita disparos múltiples dentro de la misma ventana temporal de un minuto también fue verificado y respondió correctamente.

4.3.2. Gestión de tareas concurrentes en FreeRTOS

Se verificó la correcta convivencia de las dos tareas principales del firmware: la tarea de control del dispensado y la tarea de comunicación. Para ello se ejecutó el sistema durante un período prolongado con dispensados automáticos programados cada cinco minutos, mientras la tarea de comunicación transmitía mensajes de telemetría de forma continua.

Se comprobó que la cola de mensajes `xQueueHandle` cumplió su función de sincronización entre ambas tareas, sin pérdida de eventos ni bloqueos. Durante todo el período de observación, el planificador de FreeRTOS ejecutó ambas tareas con regularidad, sin que ninguna dejara de recibir tiempo de procesamiento.

4.3.3. Protocolo de comunicación y timing de telemetría

Se verificó el protocolo de comunicación JSON entre el microcontrolador y la aplicación de escritorio. Para ello se conectó el ESP32 al STM32 mediante UART4, configurado a 115 200 baudios con formato 8N1, y se registraron los mensajes intercambiados durante una sesión de uso mediante el monitor serie de PlatformIO.

El protocolo contempla tres mensajes salientes del microcontrolador hacia la aplicación y tres comandos entrantes desde la aplicación. Los mensajes salientes son la telemetría periódica, la confirmación de dispensado y la confirmación de tara. Los comandos entrantes son el dispensado manual, la actualización de horarios y la tara remota.

La figura 4.3 muestra el registro obtenido en el monitor serie de PlatformIO durante una sesión de comunicación. Se observa la recepción del comando `set_schedules` con la lista de horarios enviada desde la aplicación, la confirmación de su recepción por parte del firmware y los mensajes de telemetría transmitidos de forma periódica, cada uno con los valores de peso y la marca temporal correspondientes.

```
[BLE->CMD] set_schedules
[BLE] Horarios recibidos:
08:00 Desayuno ACTIVO
12:00 Almuerzo ACTIVO
17:00 Merienda ACTIVO
21:30 Cena ACTIVO
[UART->STM32] {"cmd":"set_schedules","schedules":[{"hour":8,"minute":0,"label":"Desayuno","active":true},{"hour":12,"minute":0,"label":"Almuerzo","active":true},{"hour":17,"minute":0,"label":"Merienda","active":true},{"hour":21,"minute":30,"label":"Cena","active":true}]}
[UART->STM32 DBG] [COMM] Flag recibido
[DISPENSER] Enviando 4 horarios al STM32.
[UART->STM32 DBG] [COMM] Horarios actualizados: 4 recibidos.
[UART->STM32 DBG] [0] 08:00 Desayuno ACTIVO
[UART->STM32 DBG] [1] 12:00 Almuerzo ACTIVO
[UART->STM32 DBG] [2] 17:00 Merienda ACTIVO
[UART->STM32 DBG] [3] 21:30 Cena ACTIVO
[UART->STM32] {"type":"telemetry","tank_weight":0.00,"plate_weight":0.000,"datetime":{"hour":8,"minute":10,"second":25,"day":12,"month":4,"year":25,"weekday":1}}
```

FIGURA 4.3. Registro de la comunicación JSON entre la aplicación y el microcontrolador.

El período entre mensajes de telemetría consecutivos se midió con una referencia temporal externa y resultó de $1,00 \text{ s} \pm 0,02 \text{ s}$, conforme a lo especificado. No se detectaron pérdidas de mensajes ni errores de formato en los datos intercambiados durante las pruebas.

4.4. Ensayos de integración

En esta sección se presentan las pruebas del sistema completo con todos los módulos interconectados y el firmware integrado con FreeRTOS. Se describen los escenarios evaluados y los resultados obtenidos.

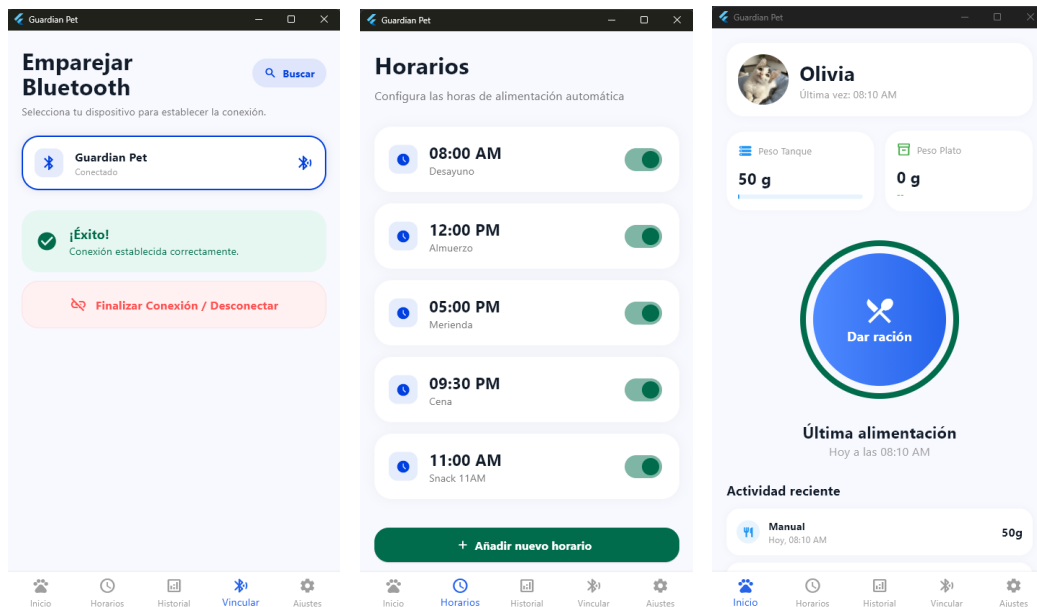
Una vez superadas las pruebas individuales, se procedió a la integración del sistema completo sobre la protoboard, con todos los componentes interconectados según el esquema de la sección 3.2.2.

La tabla 4.3 resume los escenarios evaluados, el resultado esperado y el resultado obtenido para cada caso.

TABLA 4.3. Resultados esperados vs. obtenidos en los ensayos de integración.

Escenario	Resultado esperado	Resultado
Dispensado automático por horario programado	Activación del motor y envío de mensaje <code>feeding_done</code>	Conforme
Dispensado manual por pulsador físico	Activación del motor y envío de mensaje <code>feeding_done</code>	Conforme
Monitoreo de telemetría en tiempo real	Recepción de mensaje cada 1 s con pesos actualizados	Conforme
Tara remota desde la aplicación	Tara ejecutada y confirmada por terminal	Conforme
Actualización de horarios desde la aplicación	Nuevos horarios almacenados en flash y activos	Conforme
Corte y restitución de alimentación eléctrica	Hora conservada en el DS3231 y sistema retomado	Conforme

La figura 4.4 ilustra la interfaz de la aplicación durante los ensayos de integración. Se muestran la confirmación de la vinculación Bluetooth (figura 4.4a), la configuración de horarios cargada en el dispositivo (figura 4.4b) y la pantalla principal durante el monitoreo en tiempo real (figura 4.4c), donde se observan los pesos del tanque y del plato junto con el registro de la última alimentación.



(A) Vinculación Bluetooth. (B) Configuración de horarios. (C) Monitoreo en tiempo real.

FIGURA 4.4. Interfaz de la aplicación durante los ensayos de integración.

El sistema respondió correctamente en todos los escenarios evaluados. La comunicación Bluetooth no presentó interrupciones, los eventos de dispensado se registraron con la hora correcta provista por el DS3231, y el pesaje del plato permitió detectar la variación de masa producida por el alimento dispensado.

4.5. Análisis de resultados

En esta sección se analizan los resultados obtenidos en las etapas anteriores, se identifican las limitaciones observadas y se explican los desvíos detectados respecto de lo especificado.

4.5.1. Cumplimiento de criterios de aceptación

Todos los criterios de aceptación definidos en la sección 4.1 y resumidos en la tabla 4.1 se cumplieron satisfactoriamente. El sistema de pesaje alcanzó la resolución requerida de 2 g, el motor operó sin pérdida de pasos en el rango de velocidades previsto, la hora se conservó ante cortes de energía, y la comunicación JSON no presentó errores en ninguna de las sesiones de prueba.

4.5.2. Limitaciones identificadas

Durante los ensayos se identificaron las siguientes limitaciones del prototipo:

- La resolución del sistema de pesaje de 2 g resulta adecuada para detectar el consumo de raciones mayores a 10 g, pero puede ser insuficiente para raciones pequeñas o para monitorear el consumo de agua en futuras versiones del dispositivo.
- El alcance efectivo del enlace Bluetooth Classic en el entorno de prueba fue de aproximadamente 8 m en línea de visión directa, lo que podría representar una limitación en viviendas de gran superficie.
- El ensamblado sobre protoboard con cables jumper introduce capacitancias parásitas y resistencias de contacto que no estarían presentes en un diseño con placa de circuito impreso (PCB), por lo que los resultados de consumo de corriente deben interpretarse como valores aproximados.

4.5.3. Desvíos respecto de lo planificado

El único desvío identificado respecto de lo especificado originalmente fue la variabilidad en el período de telemetría, que presentó una dispersión de $\pm 0,02$ s en lugar del valor nominal de 1 s exacto. Este desvío se atribuyó a la latencia introducida por el planificador de FreeRTOS al momento de despertar la tarea de comunicación, y no representa un inconveniente para la aplicación prevista, dado que el usuario percibe la actualización como continua.

Capítulo 5

Conclusiones

Todos los capítulos deben comenzar con un breve párrafo introductorio que indique cuál es el contenido que se encontrará al leerlo. La redacción sobre el contenido de la memoria debe hacerse en presente y todo lo referido al proyecto en pasado, siempre de modo impersonal.

5.1. Conclusiones generales

La idea de esta sección es resaltar cuáles son los principales aportes del trabajo realizado y cómo se podría continuar. Debe ser especialmente breve y concisa. Es buena idea usar un listado para enumerar los logros obtenidos.

En esta sección no se deben incluir ni tablas ni gráficos.

Algunas preguntas que pueden servir para completar este capítulo:

- ¿Cuál es el grado de cumplimiento de los requerimientos?
- ¿Cuán fielmente se pudo seguir la planificación original (cronograma incluido)?
- ¿Se manifestó algunos de los riesgos identificados en la planificación? ¿Fue efectivo el plan de mitigación? ¿Se debió aplicar alguna otra acción no contemplada previamente?
- Si se debieron hacer modificaciones a lo planificado ¿Cuáles fueron las causas y los efectos?
- ¿Qué técnicas resultaron útiles para el desarrollo del proyecto y cuáles no tanto?

5.2. Próximos pasos

Acá se indica cómo se podría continuar el trabajo más adelante.

Bibliografía

- [1] A. W. Rollins y M. Murphy. «Nutritional assessment in the cat». En: *Journal of Feline Medicine and Surgery* (abr. de 2019), págs. 442-448. URL: <https://journals.sagepub.com/doi/10.1177/1098612X19843213>.
- [2] Julie A. Churchill y Laura Eirmann. «Senior Pet Nutrition and Management». En: *Veterinary Clinics of North America: Small Animal Practice* 51.3 (mayo de 2021). Epub 2021 Feb 27. PMID: 33653535, págs. 635-651. URL: <https://doi.org/10.1016/j.cvsm.2021.01.004>.
- [3] Hikmat Yar et al. «Towards Smart Home Automation Using IoT-Enabled Edge-Computing Paradigm». En: *Sensors* 21.14 (2021), pág. 4932. URL: <https://doi.org/10.3390/s21144932>.
- [4] PetSafe. *Alimentador de mascotas automático e inteligente Smart Feed*. Consultado: mayo 2026. 2026. URL: <https://www.petsafe.com/es/p/alimentador-de-mascotas-automatico-e-inteligente-smart-feed/PFD19-16861/>.
- [5] PetSafe. *My PetSafe*. Aplicación móvil. iOS: <https://apps.apple.com/us/app/my-petsafe/id1498476188>. Android: <https://play.google.com/store/apps/details?id=net.petsafe.platform>. 2026.
- [6] PETLIBRO. *Granary 5G Wi-Fi Automatic Pet Feeder*. Consultado: mayo 2026. 2026. URL: <https://petlibro.com/products/petlibro-5g-wifi-automatic-pet-feeder>.
- [7] AVIA Semiconductor. *HX711: 24-Bit Analog-to-Digital Converter for Weigh Scales*. Hoja de datos. 2016. URL: https://cdn.sparkfun.com/datasheets/Sensors/ForceFlex/hx711_english.pdf.
- [8] Amazon Web Services. *FreeRTOS Reference Manual, Version 10.0.0*. Manual de referencia oficial. 2017. URL: https://www.freertos.org/media/2018/FreeRTOS_Reference_Manual_V10.0.0.pdf.
- [9] Texas Instruments. *UART: A Hardware Communication Protocol*. Nota de aplicación. 2020. URL: <https://www.ti.com/lit/ug/sprugp1/sprugp1.pdf>.
- [10] NXP Semiconductors. *I2C-Bus Specification and User Manual*. Especificación técnica. 2021. URL: <https://www.nxp.com/docs/en/user-guide/UM10204.pdf>.
- [11] STMicroelectronics. *NUCLEO-F446RE: STM32 Nucleo-64 Development Board*. Placa de evaluación. 2024. URL: <https://www.st.com/en/evaluation-tools/nucleo-f446re.html>.
- [12] Espressif Systems. *ESP32 Series Datasheet*. Hoja de datos. 2023. URL: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf.
- [13] Pololu Corporation. *Stepper Motor: Bipolar, 200 Steps/Rev, 42x48mm, 2.8V, 1.7A/Phase*. Hoja de datos. 2023. URL: <https://www.pololu.com/product/1200>.
- [14] Allegro MicroSystems. *A4988 DMOS Microstepping Driver with Translator and Overcurrent Protection*. Hoja de datos. 2014. URL: <https://www.allegromicro.com/~media/Files/Datasheets/A4988-Datasheet.ashx>.

- [15] Analog Devices. *DS3231: Extremely Accurate I2C-Integrated RTC/TCXO/Crystal*. Hoja de datos. 2015. URL: <https://www.analog.com/media/en/technical-documentation/data-sheets/ds3231.pdf>.
- [16] STMicroelectronics. *STM32CubeIDE Integrated Development Environment*. Entorno de desarrollo integrado. 2024. URL: <https://www.st.com/en/development-tools/stm32cubeide.html>.
- [17] Doxygen. *Doxygen: Generate Documentation from Source Code*. Herramienta de documentación de código fuente. 2024. URL: <https://www.doxygen.nl>.
- [18] PlatformIO Labs. *PlatformIO: A Professional Collaborative Platform for Embedded Development*. Entorno de desarrollo integrado. 2024. URL: <https://platformio.org>.
- [19] Google LLC. *Flutter: Build Apps for Any Screen*. Framework de desarrollo de aplicaciones multiplataforma. 2024. URL: <https://flutter.dev>.
- [20] Bluetooth SIG. *Serial Port Profile (SPP)*. Consultado: mayo 2026. 2003. URL: <https://www.bluetooth.com/specifications/specs/serial-port-profile-1-1/>.
- [21] Texas Instruments. *LM2596 Series: SIMPLE SWITCHER Power Converter 150 kHz 3A Step-Down Voltage Regulator*. 2020. URL: <https://www.ti.com/lit/ds/symlink/lm2596.pdf>.
- [22] A. y col. Rodríguez. «Diseño y construcción de un sistema de control de micropasos usando modulación de ancho de pulso». En: *Revista Ciencia e Ingeniería* 35.1 (2014).
- [23] JSON.org. *Introducing JSON*. Consultado: mayo 2026. 2023. URL: <https://www.json.org/json-en.html>.
- [24] Amazon Web Services. *Semaphore / Mutex API – FreeRTOS Kernel*. <https://www.freertos.org/Real-time-embedded-RTOS-mutexes.html>. Consultado en junio de 2026. 2024.
- [25] Saleae. *Logic 2 Software and Logic Analyzers*. 2024. URL: <https://www.saleae.com>.